

OSSP – PRACTICAL 4

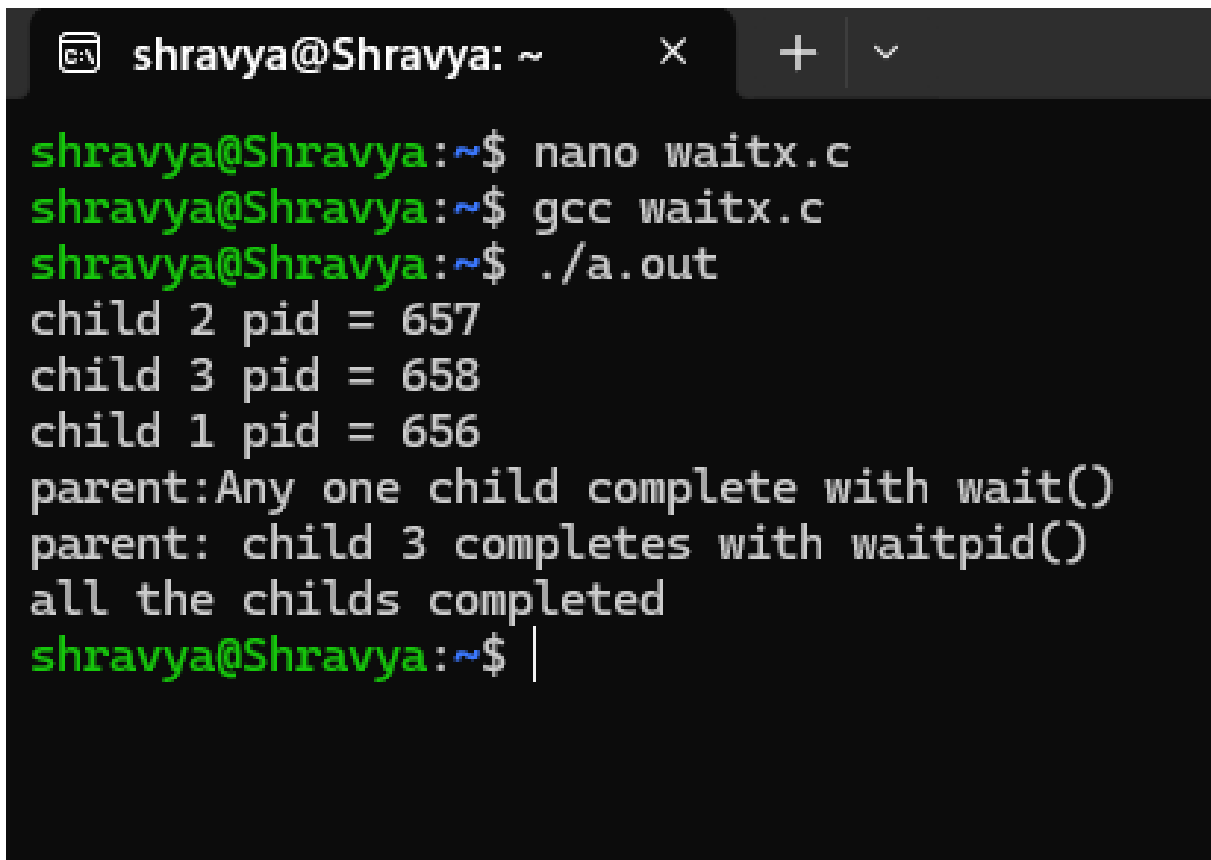
Process Synchronization using wait() and waitpid() and Zombie Process Handling

1. Aim

To create multiple child processes and synchronize their completion using wait() and waitpid(), compare their behavior, and demonstrate zombie process handling.

2. Output – Multiple Child Processes

The following terminal output shows compilation and execution of the wait/waitpid program. Three child processes are created and the parent waits for their completion.



```
shravya@Shravya: ~  
shravya@Shravya:~$ nano waitx.c  
shravya@Shravya:~$ gcc waitx.c  
shravya@Shravya:~$ ./a.out  
child 2 pid = 657  
child 3 pid = 658  
child 1 pid = 656  
parent:Any one child complete with wait()  
parent: child 3 completes with waitpid()  
all the childs completed  
shravya@Shravya:~$ |
```

Observation: Three child processes are created with different PIDs. The parent uses wait() to wait for a child and waitpid() to specifically wait for child 3. Finally, all children are completed.

3. Output – Zombie Process Handling

The following terminal output demonstrates the parent-child execution used for zombie process handling.

```
shravya@Shravya:~$ nano zombie.c
shravya@Shravya:~$ gcc zombie.c
shravya@Shravya:~$ ./a.out
parent process 766
child process 767
parent process completed
shravya@Shravya:~$ |
```

Observation: The child process completes first, and the parent process then completes after synchronization.

4. wait() vs waitpid()

Function	Behavior
wait()	Waits for any one child process to terminate.
waitpid()	Waits for a specific child process using its PID.

5. Result

Multiple child processes were successfully created and synchronized using wait() and waitpid(). The difference between both functions was observed, and proper parent-child synchronization was demonstrated for zombie process handling.